

[Edit](#)

Build Features by Using Copilot in Python

Challenge Overview

Understand the scenario

You are a Developer at Hexelo using GitHub Copilot in Visual Studio Code to build and expand Python program features from comments and prompts.

In this Challenge Lab, you will build features by using Copilot in Python. First, you will generate starter code from a comment. Next, you will collect user input, and then you will save data to a file. Finally, you will use Copilot to extend functionality with menu-driven features.

 You must complete the **Set Up Git and Connect to GitHub** setup lab before starting any challenge in this series. The set-up lab should only be completed once.

Navigating the Challenge Lab

Unable to process content from <https://raw.githubusercontent.com/LODSContent/Challenge-V3-Framework/main/Templates/Environments/None.md>

 ► Quick tips for navigating the Challenge Lab instructions.

[New to Challenge Labs? Click here to learn more.](#)

Generate starter code from a comment

Hints Enabled

No Yes

 You must complete the **Set Up Git and Connect to GitHub** setup lab before starting any challenge in this series. The set-up lab should only be completed once.

- Sign in to the virtual machine as **LabUser** by using  **Passw0rd!** as the password.
- Enter your GitHub Username, Email Address, and Password into the following associated textboxes:

GitHub Username

GitHub Email Address

GitHub Password

- Open **Microsoft Edge**, go to  <https://github.com>, and then sign in as  **<GithubUN>** by using  **<GithubPW>** as the password.

 Expand this hint for guidance on signing in to a GitHub account.

- On the Windows taskbar, in Search, enter **Edge**, and then select **Microsoft Edge** to open a new browser.
- In the Address bar, enter  <https://github.com>, and then press **Enter**.
- On the GitHub home page, in the upper right-hand corner, select **Sign in**.
- On the *Sign in to GitHub* page, in Username or email address, sign in as  [<GithubUN>](#) using  [<GithubPW>](#) as the password.
- If presented with the *Save password* dialog, select **Never**.
- On the *Device verification* page, enter the **Device Verification Code** sent to the email address used to register the account.

 Leave the **GitHub** browser window open.

- In  [Visual Studio Code](#), create a project folder named  [copilot_items](#) in  [D:\LabFiles](#), and then create a new file named  [main.py](#).

 Expand this hint for guidance on creating a project and file.

- On the Windows taskbar, in Search, enter  [Visual Studio Code](#), and select it to open **VS Code**.
- Close the two *Welcome* tabs.
- In **VS Code**, select **File**, and then select **Open Folder...**
- Navigate to  [D:\LabFiles](#), select *New folder*, enter  [copilot_items](#) to create a folder, double-click on the new folder to open it, and then select **Select Folder**.
- On the *Do you trust the authors of the files in this folder?* dialog, select **Yes, I trust the authors**.
- In the Explorer, in *copilot_items*, select the **Add file** icon.
- Enter  [main.py](#), and then press **Enter**.

 Want to learn more? Review the documentation on [creating and opening files in VS Code](#).

- Verify that the following extensions have been installed in **Visual Studio Code**:  [GitHub Copilot](#),  [GitHub Copilot Chat](#), and  [Python extension for Visual Code](#).

💡 Expand this hint for guidance on verifying extensions in Visual Studio Code. ^

- Select the **Extensions** icon in **VS Code**.



- Search for, and if necessary, install the extensions below:
 - GitHub Copilot
 - GitHub Copilot Chat
 - Python extension for Visual Code
- Select **Ctrl+Shift+P**, and then enter Copilot.
- Confirm commands appear.

💡 Want to learn more? Review the documentation on [setting up GitHub Copilot in VS Code](#).

- Confirm Python is installed by running the python command in the **VS Code** Terminal.

💡 Expand this hint for guidance on confirming Python installation. ^

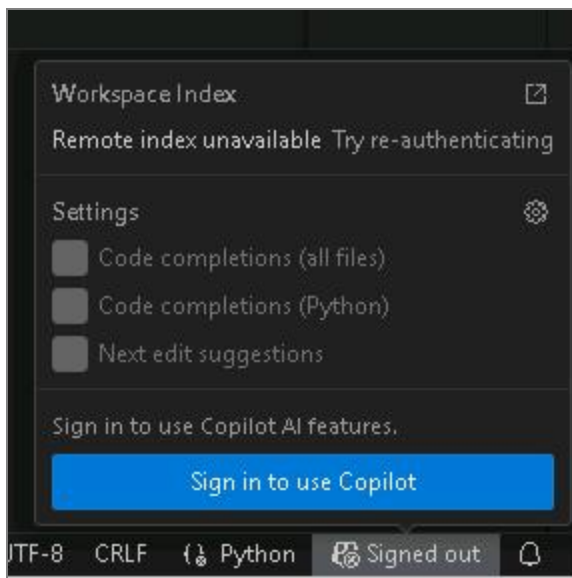
- Select **Terminal**, and then select **New Terminal**.
- Run the following command:

```
bash  
python --version
```

- Ensure a valid Python version appears.

💡 Want to learn more? Review the documentation on [running Python in VS Code](#).

- In the bottom right-hand corner, select **Signed out** to sign in to Copilot and authorize **Visual Studio Code**.



- In the **main.py** file, enter the following comment:

📄 *# Build a small items tracker: ask for user input, store items in a list, save to a file, and provide a text menu.*

💡 Expand this hint for guidance on adding a descriptive comment. ^

- In **main.py**, at the top of the file, copy the comment supplied above and paste it into the file.
- Press **Enter** to place the cursor on the next line.

💡 Want to learn more? Review the documentation on [writing effective Copilot prompts](#).

- Generate code by using Copilot's suggestion starting with `T` `def main()`.

💡 Expand this hint for guidance on generating code using Copilot. ^

- On the next line, enter `T` `def`, add a space, and start writing a function named `T` `main()`.
- Wait for Copilot's ghost text suggestion.
- Select **Tab** to accept it, or continue typing to refine it.

💡 Want to learn more? Review the documentation on [getting code suggestions in your IDE with GitHub Copilot](#).

📄 Your script may vary slightly from the example:

```
main.py
main.py
1 # Build a small items tracker: ask for user input, store items in a list, save to a file, and provide a
2 def main():
3     import os
4
5     items = []
6     filename = "items.txt"
7
8     def load_items():
9         if os.path.exists(filename):
10             with open(filename, "r") as f:
11                 return [line.strip() for line in f.readlines()]
12             return []
13
14
```

📄 If you receive an error, or if nothing appears, add this docstring prompt: ^

python

```
def main():  
    import os  
  
    items = []  
    filename = "items.txt"  
  
    def load_items():  
        if os.path.exists(filename):
```

⚠ Copilot will continue to generate the methods based on the comment, but stop after Copilot generates the *load_items* function.

- Review and save the **main.py** file.

💡 Want to learn more? Review the documentation on [saving and managing files in VS Code](#).

Check your work

Verify

Collect and store user input

Hints Enabled

No Yes

- Create an empty `list` and a sanitization helper function named `clean` in the **main.py** file by using the following code:

python

```
items: list[str] = []

def clean(text: str) -> str:
    return text.strip()
```



Expand this hint for guidance on creating an empty list and a sanitization helper.



- Add the code supplied above near the top of the **main.py** file, below the comment, but above other functions.
- Add the following `ask_item` function above the `clean` function to capture input by using the following code:

python

```
def ask_item() -> str:
    raw = input("Enter an item: ")
    return clean(raw)
```


💡 Expand this hint for guidance on adding a function to capture input. ^

- Copy the code supplied above and add it to the **main.py** file.

```
2
3 items: list[str] = []
4
5 def ask_item() -> str:
6     raw = input("Enter an item: ")
7     return clean(raw)
8
9 def clean(text: str) -> str:
10    return text.strip()
11
```

- Ensure there is a blank line between the two functions.

💡 Want to learn more? Review the documentation on [input and output in Python](#).

- Add an **add_item** function to append input to the list, and then display the results by using the following code:

python

```
def add_item(items: list[str]) -> None:
    item = ask_item()
    if not item:
        print("No input provided.")
        return
    if item in items:
        print("Item already exists.")
        return
    items.append(item)
    print(f"Added: {item}")
    print(f"Current items: {items}")
```

💡 Expand this hint for guidance on appending input and displaying results. ^

- Copy the **add_item** function from the code block supplied above and add it at the bottom of the **main.py** file.

💡 Want to learn more? Review the documentation on [using lists in Python](#).

- Prevent empty or duplicate entries by **temporarily** adding the following code to the bottom of the file:

```
python  
  
if __name__ == "__main__":  
    add_item(items)  
    add_item(items)
```

 Want to learn more? Review the documentation on [testing code blocks in Python](#).

- Ask Copilot to **Fix and optimize the entire script**, and then save the **main.py** file.

 Expand this hint for guidance on asking Copilot to fix and optimize the script. 

- Save the **main.py** file, and then in *Ask Copilot*, enter **Fix and optimize the entire script**.
- Select the entire **main.py** script, copy it, and then paste it on the next line of the *Ask Copilot* prompt textbox.
- Select **Send**.
- Review Copilot's suggestion, select **Apply to main.py**, and then select **Keep**.
- **Save** the file.

- Test the **main.py** script by using the **python** command.

 Expand this hint for guidance on running a Python script. 

- In **VS Code**, select **Terminal**, and then select **New Terminal**.
- In the **VS Code** Terminal, enter **python main.py**, and then press **Enter**.
- Ensure you are in the **copilot_items** directory. If not, enter **cd D:\LabFiles\copilot_items**, and then run the **python main.py** command again.

- Add **test item #1** using the menu, and then exit the program.

💡 Expand this hint for guidance on adding an item. ^

- In the **VS Code** Terminal, enter the number for the **Add item** option (likely **T 1**), and then press **Enter**.
- When prompted to *Enter an item*, enter **T test item #1**, and then press **Enter**.
- Enter the number for **Exit** or **Quit** (likely **T 3**), and then press **Enter**.

- After confirming that empty and duplicate entries are blocked, remove the temporary `if __name__ == "__main__":` block of code from the **main.py** file.

python

```
if __name__ == "__main__":  
    add_item(items)  
    add_item(items)
```

Check your work

Verify

Save items to a file using Python

Hints Enabled

No Yes

- Add the `Pathlib` import and write a `save_to_file` function by using the code below.

python

```
def save_to_file(items: list[str], path: Path | str = "items.txt") -> Path:
    p = Path(path)
    with p.open("w", encoding="utf-8") as f:
        for it in items:
            f.write(f"{it}\n")
    return p
```

💡 Expand this hint for guidance on writing the `save_to_file` function. ^

- At the top of `main.py`, but under the comment, add the following:

python

```
from pathlib import Path
```

- Add one blank line, and then on the next line, add the function from above.

💡 Want to learn more? Review the documentation on [working with files and paths in Python](#).

- Using the code below, add a `show_file` function to confirm the writing of items by reading the file back.

python

```
def show_file(path: Path | str = "items.txt") -> None:
    p = Path(path)
    if not p.exists():
        print("No file to show.")
        return
    print("File contents:")
```

```
print(p.read_text(encoding="utf-8"))
print("")
```



Want to learn more? Review the documentation on [reading and writing files in Python](#).

- Perform a quick manual test by using the following code temporarily:

python

```
if __name__ == "__main__":
    items.extend(["alpha", "beta"])
    save_to_file(items)
    show_file()
```



Want to learn more? Review the documentation on [basic file I/O operations in Python](#).

- Ask Copilot to **T** Fix and optimize the entire script..
- Run **T** python main.py, and then verify that the output file appears and that the lines are correct. Your results should be similar to the screenshot below.

```
PS D:\LabFiles\copilot_items> python main.py
File contents:
alpha
beta
```

- Once the output is correct, remove the temporary `if __name__ == "__main__":` block so the final program uses a single entry point.

python

```
if __name__ == "__main__":
    items.extend(["alpha", "beta"])
    save_to_file(items)
    show_file()
```



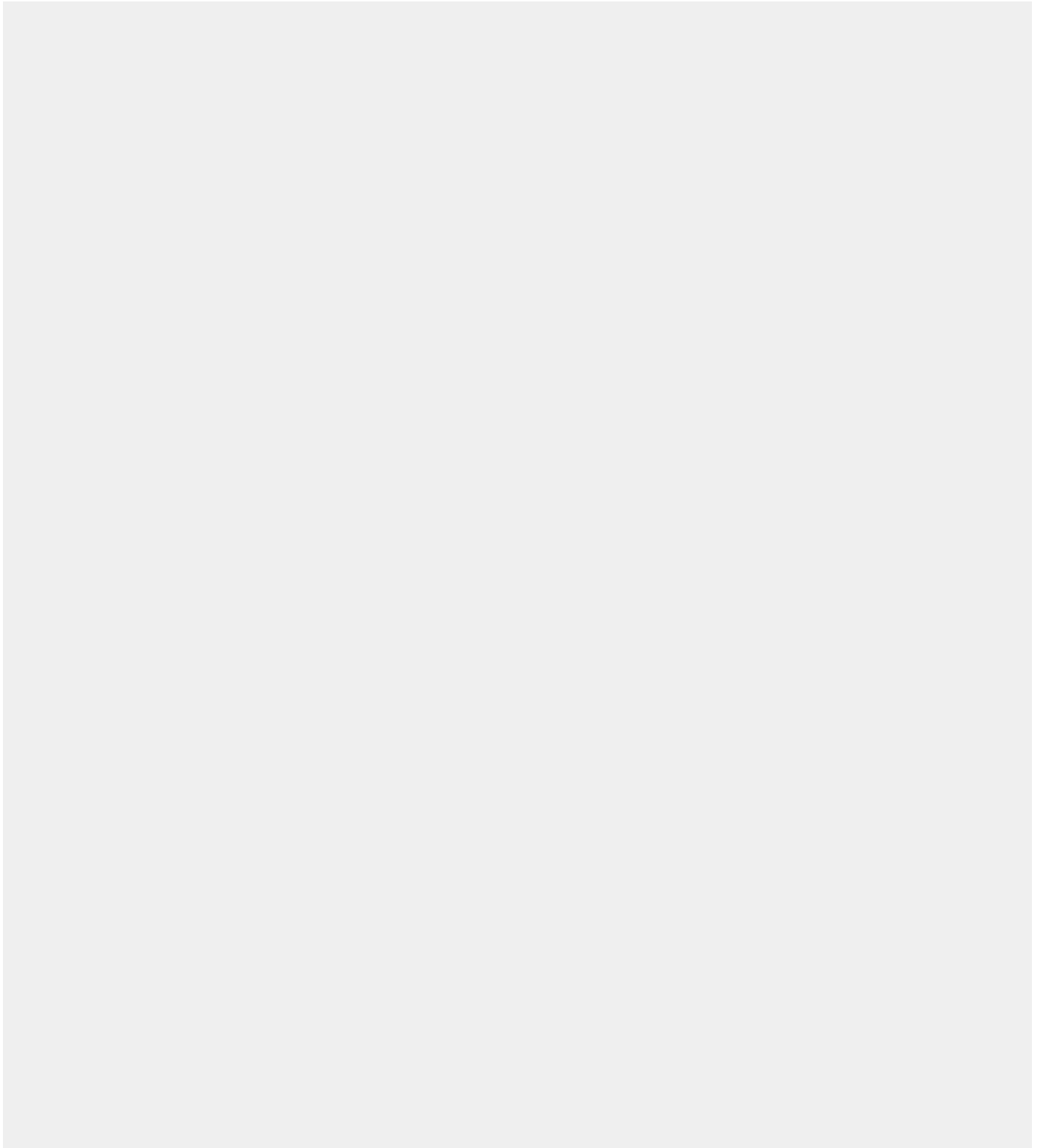
Alternatively, you could ask Copilot the request below, instead of manually editing the script.

T Remove the temporary if **name** == "main": block afterward so the final program uses a single entry point.

Check your work



Verify



Build a menu and extend features using Copilot

Hints Enabled

No Yes

- Implement a simple list display by adding a function named `list_items` using the following code. Add the `list_items` function above the `main` function and below the other functions.

python

```
def list_items(items: list[str]) -> None:
    if not items:
        print("No items yet.")
        return
    for i, it in enumerate(items, start=1):
        print(f"{i}. {it}")
```



Want to learn more? Review the documentation on [enumerate and loops in Python](#).

- Build the `main` menu loop by replacing the `main` function with the one supplied below:

python

```
# Menu: [A]dd [L]ist [S]ave [Q]uit

def main() -> None:
    while True:
        print("\nMenu: [A]dd [L]ist [S]ave [Q]uit")
        choice = input("Select an option: ").strip().lower()
        if choice in ("a", "add"):
            add_item(items)
        elif choice in ("l", "list"):
            list_items(items)
        elif choice in ("s", "save"):
            save_to_file(items)
            print("Saved to items.txt")
        elif choice in ("q", "quit"):
            print("Goodbye.")
            break
        else:
            print("Invalid option.")
```

 Want to learn more? Review the documentation on [writing loops and conditionals in Python](#).

- Add the following entry point at the bottom of the file (if it is not already present):

```
python
if __name__ == "__main__":
    main()
```

- Ensure there is only one `if __name__ == "__main__":` block in the final script.
- Use Copilot to add an extra feature and create a `delete_item` function by adding the following code:

```
python
# Feature: delete an item by number or name

def delete_item(items: list[str]) -> None:
    if not items:
        print("No items to delete.")
        return
    list_items(items)
    target = input("Enter number or exact name to delete: ").strip()
    if target.isdigit():
        idx = int(target) - 1
        if 0 <= idx < len(items):
            removed = items.pop(idx)
            print(f"Removed: {removed}")
            return
        print("Invalid number.")
        return
    if target in items:
        items.remove(target)
        print(f"Removed: {target}")
    else:
        print("Not found.")
```

 Expand this hint for guidance on using Copilot to add an extra feature. 

- Above the **menu** (or **main**) function, add the comment and function (accept Copilot's version if similar) supplied above.

 Want to learn more? Review the documentation on [prompting Copilot to extend code features](#).

- Update the menu header comment by using the following:

python

 `# Menu: [A]dd [L]ist [S]ave [D]elete [Q]uit`

- Wire the delete feature into the menu loop by adding the following code above the  `quit` statement:

python



```
elif choice in ("d", "del", "delete"):
    delete_item(items)
```

 Confirm that **items** is defined once at the top of the file ( `items: list[str] = []`) so that all menu actions, including delete, operate on the same list.

- Ask Copilot to  `Fix and optimize the entire script.`
- Run a final program test by using  `python main.py`, add the following items, save, view the list, and then quit.
 -  `laptop`
 -  `server`
 -  `printer`

 Want to learn more? Review the documentation on [running Python programs in VS Code](#).

 If you receive an error that resembles **expected an indented block after 'if' statement on line ###**, copy the error into the Copilot prompt textbox, and ask Copilot to fix it for you.



```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS pwsh - copilot_items + - [ ] [ ] ... ^
PS D:\LabFiles\copilot_items> python main.py
File "D:\LabFiles\copilot_items\main.py", line 104
    if __name__ == "__main__":
IndentationError: expected an indented block after 'if' statement on line 104
PS D:\LabFiles\copilot_items> [ ]
```

Check your work



Summary

Congratulations, you have completed the **Build Features by Using Copilot in Python** Challenge Lab.

You have accomplished the following:

- Generated starter code from a comment.
- Collected and stored user input.
- Saved items to a file using Python.
- Built a menu and extended features using Copilot.

Ending your lab

To ensure your lab is recorded as complete, select **Submit** or **End** below. Exiting [X] the lab will result in an incomplete status.

 Once you select **Submit** or **End**, you will not be able to return to this Challenge Lab.

Your feedback is important!

As you end your Challenge Lab, please take a few minutes to complete the short survey that will appear in the next window. Alternatively, you may provide your feedback directly to [Challenge Labs feedback](#).

Looking for your next Challenge Lab?

Below are recommended Challenge Labs to try next.

- Set Up Git and Connect to GitHub [Prerequisite Lab]
- Create a Class from a Prompt Using GitHub Copilot [Guided]
- Build Features by Using Copilot in Python [Guided]
- Write Unit Tests Automatically by Using Copilot [Guided]
- Add Comments and Docstrings by Using Copilot [Guided]
- Develop an API Client with GitHub Copilot [Guided]

- Refactor Python Code by Using Copilot [Guided]
- Create Shell Scripts by Using Copilot [Guided]
- Automate CI Workflows by Using GitHub Actions [Guided]
- Document Projects by Using Markdown and Copilot [Guided]
- Refactor and Optimize Code with GitHub Copilot [Guided]
- Build Command-Line Tools by Using Copilot [Guided]
- Utilize Copilot for Python Code Correction [Guided]

